

# Sorting as Gradient Flow on the Permutohedron

Jonathan Robert Landers  
jonathan.robert.landiers@gmail.com

## Abstract

We investigate how sorting algorithms navigate the complexity of permutation space. The main contribution is a geometric and dynamical model of comparison sorting using the permutohedron, in which rank words move through several related geometries: adjacent-swap paths along the 1-skeleton, comparison half-spaces that shrink the feasible vertex set, and a continuous ambient relaxation that flows toward the sorted word  $v_s = (1, 2, \dots, n)$ . This model reframes the classical  $\Omega(n \log n)$  lower bound: comparisons appear as information-bearing constraints on candidate rank words, while the continuous flow makes contraction of disorder explicit. To support this analysis, we present information-theoretic and structural perspectives, including decision-tree arguments, adjacent-swap paths with  $\Theta(n^2)$  behavior, and linear constraints that carve through the permutohedron. Together, these viewpoints show how comparisons shape the feasible space as a whole, why local motion gives the quadratic regime, and how straight-line ambient descent gives a benchmark for contraction toward the sorted state. This geometric perspective connects to broader questions in theoretical computer science, such as whether the existence of structure can explain why certain computational problems permit efficient solutions.

**MSC 2020:** 68Q25, 52B11, 90C52 **Keywords:** sorting algorithms, permutohedron, gradient flow, computational complexity

## 1 Introduction

Consider the fundamental problem of sorting a list

$$L = [\ell_1, \ell_2, \dots, \ell_n]. \quad (1)$$

The elements are taken to lie in a totally ordered set so that their relative order is well-defined. Rather than viewing sorting merely as a sequence of symbolic operations, it is useful to regard it as a problem of navigating a large structured space: each input corresponds to an unknown permutation in a space of  $n!$  possibilities, and the task of sorting is to locate the correct ordering within this space using comparisons. From this geometric standpoint, sorting becomes motion through a shaped space rather than only a symbolic procedure. Discrete combinatorial reasoning unfolds within a geometric landscape whose constraint structure governs how efficiently algorithms can move, and algorithms succeed to the extent that their paths align with the structure of the space they inhabit.

**Notation.** We keep four related objects separate. The integer  $i$  is the original index of an item, and  $\ell_i$  is the value at that index. Since the entries are assumed pairwise distinct, define the original rank map

$$\rho(i) = \text{rank}(\ell_i), \quad (2)$$

where rank 1 is smallest and rank  $n$  is largest. At algorithmic time  $t$ , let  $p_t(k)$  be the original index of the item currently occupying array position  $k$ . The rank word plotted on the permutohedron is

$$x_t(k) = \rho(p_t(k)). \quad (3)$$

Thus  $x_t$  records the ranks currently sitting in array positions, and initially

$$x_0 = (\rho(1), \rho(2), \dots, \rho(n)). \quad (4)$$

Sorting moves the rank word  $x_0$  toward  $v_s = (1, 2, \dots, n)$ . The sorted index order is a different object:  $\sigma(k)$  is the original index of the item occupying sorted position  $k$ , so  $\sigma = \rho^{-1}$  and

$$[\ell_{\sigma(1)}, \ell_{\sigma(2)}, \dots, \ell_{\sigma(n)}] \quad (5)$$

is sorted. For example, if  $L = [3, 1, 2]$ , then  $\rho = (3, 1, 2)$  but  $\sigma = (2, 3, 1)$ . These differ in general, even though for  $L = [5, 4, 2]$  both may be written as 321.

We write  $\tau$  for a permutation of the index set  $\{1, 2, \dots, n\}$ , acting on  $L$  by

$$\tau(L) = [\ell_{\tau(1)}, \ell_{\tau(2)}, \dots, \ell_{\tau(n)}]. \quad (6)$$

**Setting.** A brute-force baseline is to enumerate all  $n!$  permutations  $\tau \in S_n$  and select the one that yields the sorted arrangement  $\sigma$ . This costs  $\Theta(n \cdot n!)$ , since each candidate ordering requires  $O(n)$  time to check. We use standard asymptotic notation as in [1, 2]; bounds are worst-case unless stated otherwise, randomized costs are denoted by  $\mathbb{E}[T(n)]$ , and all logarithms are base 2.

Within this framework, established sorting algorithms such as MergeSort, HeapSort, and QuickSort achieve a remarkable improvement, reducing the search from factorial to log-linear scaling. By contrast, simpler local-swap methods such as BubbleSort make only adjacent comparisons and this locality restriction yields a quadratic  $\Theta(n^2)$  runtime and offers an intermediate improvement between the brute-force  $n!$  baseline and the optimal  $\Theta(n \log n)$  regime. MergeSort and HeapSort guarantee  $O(n \log n)$  worst-case performance, while QuickSort attains the same bound in expectation. Each exploits structure in the input, effectively pruning large regions of permutation space.

This striking efficiency has long inspired curiosity about the connection between structure and computational tractability. Foundational work by Knuth [3], Shannon [4], and others, along with recent combinatorial and geometric perspectives [5, 6, 7], reflects ongoing efforts to understand how structure constrains and organizes computation.

The main innovation presented in this paper is a continuous-time formulation of sorting as an ambient gradient flow on the permutohedron, placed alongside the discrete geometries induced by sorting operations themselves. Adjacent swaps determine a local graph geometry, comparisons determine an information geometry on feasible rank words, and the Euclidean flow supplies a smooth benchmark for contraction. This framework gives a geometric and dynamical reframing of the classical  $\Omega(n \log n)$  lower bound for comparison-based sorting without replacing the decision-tree proof. Rather than treating efficiency as an incidental property of algorithms, this formulation exposes it as a consequence of geometry: local moves walk through the polytope, while global comparisons carve and organize the feasible space.

We begin with the decision-tree lower bound to set the complexity scale, then reinterpret comparisons as linear constraints acting on the permutohedron, and finally introduce a gradient-flow model that makes geometric contraction explicit.

## 2 Information-Theoretic Decision-Tree Lower Bound

The lower bound for any comparison-based sorting algorithm is first established by modeling the process as a binary decision tree, whose leaves represent permutations of the index set applied to a fixed input list.

**Lemma 2.1** (Decision-Tree Lower Bound). *Let  $T$  be a binary decision tree that correctly sorts  $n$  elements using comparisons. Then the height  $h$  of  $T$  satisfies*

$$h \geq \log(n!). \quad (7)$$

*Proof.* A deterministic comparison-based algorithm must yield a different sequence of comparisons for each of the  $n!$  possible orderings of the input. Consequently, the decision tree must have at least  $n!$  leaves. Because any binary tree of height  $h$  has at most  $2^h$  leaves, we obtain:

$$2^h \geq n!. \quad (8)$$

Taking the base-2 logarithm of both sides yields:

$$h \geq \log(n!). \quad (9)$$

Using Stirling's approximation,

$$\log(n!) = n \log n - O(n), \quad (10)$$

so that  $h = \Omega(n \log n)$ . This lower bound is tight for comparison-based sorting algorithms.  $\square$

Interestingly, Harris, Kretschmann, and Martínez Mori [6] have shown that the total number of comparisons made by QuickSort over all  $n!$  input orderings coincides with the number of parking-preference lists admitting exactly  $n - 1$  lucky cars. Their result offers an enumerative counterpart to the information-theoretic viewpoint and further underscores how global comparison structure governs the complexity of sorting.

Figure 1 illustrates the decision-tree framework through a complete binary tree for sorting three elements, instantiated with the input  $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$ . The leaves are labeled by candidate sorting permutations of the original indices, and the corresponding output at a leaf is the reordered list. The highlighted root-to-leaf path records the three comparisons performed on this input and terminates at the leaf labeled  $\sigma = (3, 2, 1)$ , which yields the sorted output  $[2, 4, 5]$ . Thus the example realizes  $\lceil \log(3!) \rceil = 3$ , in agreement with the lower bound. Knuth's seminal analysis established this information-theoretic scale for sorting algorithms [3].

The decision-tree model fixes the comparison scale, but it does not explain the geometry of the space being carved. This distinction matters: algorithms restricted to local adjacent comparisons, such as BubbleSort, resolve one inversion at a time and have worst-case cost  $\Theta(n^2)$ , while arbitrary comparisons can approach the balanced  $\Theta(n \log n)$  scale. In the next section we make this contrast geometric by interpreting comparison outcomes as linear inequalities on the permutohedron. Sorting then appears not only as logical branching but as structured contraction of the feasible rank words until a single vertex remains.

### 3 Geometric Perspective: The Permutohedron and Half-Space Constraints

We now turn to a geometric interpretation of sorting on the permutohedron. Sorting is first viewed as a path traversal along the edges of this polytope, where adjacent swaps move step-by-step between neighboring permutations and lead to the familiar  $\Theta(n^2)$  behavior of local sorting algorithms. The perspective then shifts and comparisons are reinterpreted as linear inequalities that cut through the permutohedron, geometrically representing the improved  $\Theta(n \log n)$  scale. With this interpretation, sorting is no longer merely a walk on edges but a process of geometric contraction, in which successive half-space constraints rapidly isolate the sorted order as a single vertex.

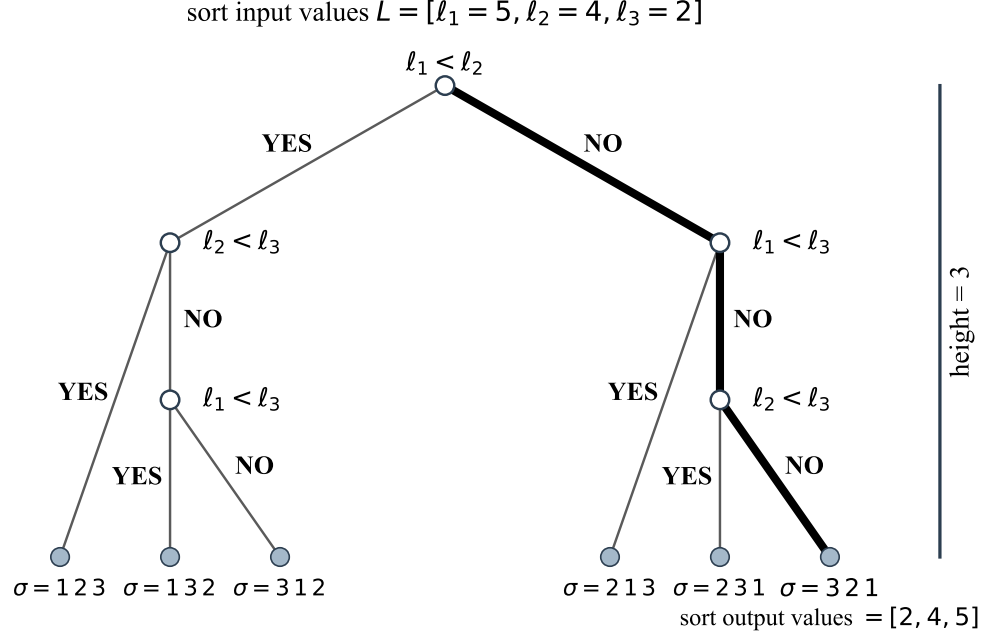


Figure 1: Decision tree for sorting  $L = (\ell_1 = 5, \ell_2 = 4, \ell_3 = 2)$ . Internal nodes are comparisons and leaves are candidate sorting permutations. The highlighted path identifies  $\sigma = (3, 2, 1)$ , yielding  $[2, 4, 5]$  in three comparisons, matching  $\lceil \log(3!) \rceil = 3$ . Later permutohedron figures use rank-word coordinates rather than these index-order leaf labels.

**Definition 3.1** (Permutohedron). Let

$$v_s = (1, 2, \dots, n) \in \mathbb{R}^n. \quad (11)$$

The permutohedron  $\mathcal{P}_n$  is defined as

$$\mathcal{P}_n = \text{conv}\{\pi(v_s) \mid \pi \in S_n\}. \quad (12)$$

Its vertex set is  $\mathcal{V}_n = \{\pi(v_s) \mid \pi \in S_n\}$ . In this paper these vertices are read as rank words: the coordinate in position  $k$  is the rank currently occupying that position.

**Proposition 3.2** (Dimensionality and Affine Containment). *The permutohedron  $\mathcal{P}_n$  is an  $(n-1)$ -dimensional polytope contained in the affine hyperplane*

$$H = \left\{ x \in \mathbb{R}^n \mid \sum_{i=1}^n x_i = \frac{n(n+1)}{2} \right\}. \quad (13)$$

*Proof.* For any permutation  $\pi \in S_n$ ,

$$\sum_{i=1}^n (\pi(v_s))_i = \sum_{i=1}^n i = \frac{n(n+1)}{2}. \quad (14)$$

Thus, every vertex of  $\mathcal{P}_n$  lies in the hyperplane  $H$ .

To see that these vertices span  $H$ , note that for each adjacent transposition  $(i, i+1)$ ,

$$(i, i+1)(v_s) - v_s = \pm(e_{i+1} - e_i), \quad (15)$$

with the sign depending only on the convention for writing the permutation action. The vectors  $e_{i+1} - e_i$  for  $i = 1, \dots, n-1$  form a basis of the subspace

$$\left\{ x \in \mathbb{R}^n : \sum_{i=1}^n x_i = 0 \right\}. \quad (16)$$

Since adjacent transpositions generate all permutations, it follows that the set  $\{\pi(v_s) : \pi \in S_n\}$  affinely spans the entire hyperplane  $H$ .

Because  $H$  has dimension  $n-1$ , the permutohedron  $\mathcal{P}_n$  is an  $(n-1)$ -dimensional polytope.  $\square$

### 3.1 Adjacent-Swap Walks and Their Geometric Interpretation

Before introducing the linear-constraint formulation, it is helpful to visualize how rank-word vertices relate to one another on the permutohedron  $\mathcal{P}_n$ . Because adjacent transpositions generate  $S_n$ , they appear naturally as edges of the 1-skeleton (Cayley graph) of  $\mathcal{P}_n$ . Traversing these edges corresponds to resolving one inversion at a time and gives a literal geometric realization of sorting as motion on a polytope, mirroring the stepwise progression seen in the decision-tree example for the input  $L = (\ell_1, \ell_2, \ell_3) = (5, 4, 2)$ .

Figure 2 depicts this adjacent-swap path for  $n = 3$  in two complementary forms: a flattened 2-dimensional schematic and a full 3-dimensional embedding of  $\mathcal{P}_3$ . In both views, the highlighted trajectory connects the reverse rank word  $(3, 2, 1)$  to the sorted rank word  $(1, 2, 3)$  by successive neighbor exchanges. These local moves provide a geometric model for the  $\Theta(n^2)$  behavior of adjacent-swap sorting algorithms. The low-dimensional example is used only because  $\mathcal{P}_3$  can be drawn directly; the definitions, constraints, and flow estimates below are stated for arbitrary  $n$ .

This path-based perspective sets the stage for the transition to hyperplane cuts below. When comparison outcomes are interpreted as linear inequalities  $x_a < x_b$  between current-position ranks, they define half-spaces whose intersection with  $\mathcal{P}_n$  produces a much faster, globally contracting process. As shown in Proposition 3.3, these constraints prune the feasible rank words at a rate consistent with the information-theoretic lower bound of  $\Omega(n \log n)$  comparisons, producing an exponential narrowing of the search space.

### 3.2 Linear Constraints and Global Geometric Collapse

**Proposition 3.3** (Comparison Constraints and Feasible Rank Words). *Consider comparison outcomes represented in rank-word coordinates as inequalities of the form*

$$x_a < x_b \quad (17)$$

*between ranks in current positions. Any comparison decision tree that correctly isolates one sorted outcome for each of the  $n!$  candidate rank words  $\mathcal{V}_n$  has worst-case depth at least  $\log(n!) = \Omega(n \log n)$ . Conversely, optimal comparison sorts provide  $O(n \log n)$  algorithmically chosen comparison constraints along each input path, distinguishing a unique feasible order type. When the identified order is read in the algorithm's final output coordinates, this feasible rank word is the sorted vertex  $v_s = (1, 2, \dots, n)$ .*

*Proof.* At a given decision node, the algorithm compares the two items currently occupying positions  $a$  and  $b$ . In the notation above, these are the original-indexed items  $p_t(a)$  and  $p_t(b)$ . If the outcome is  $\ell_{p_t(a)} < \ell_{p_t(b)}$ , then in rank-word coordinates it is recorded as  $x_t(a) < x_t(b)$ ; the opposite comparison outcome is recorded as  $x_t(b) < x_t(a)$ . Thus each comparison outcome selects one of two affine half-spaces for the candidate current-position rank words.

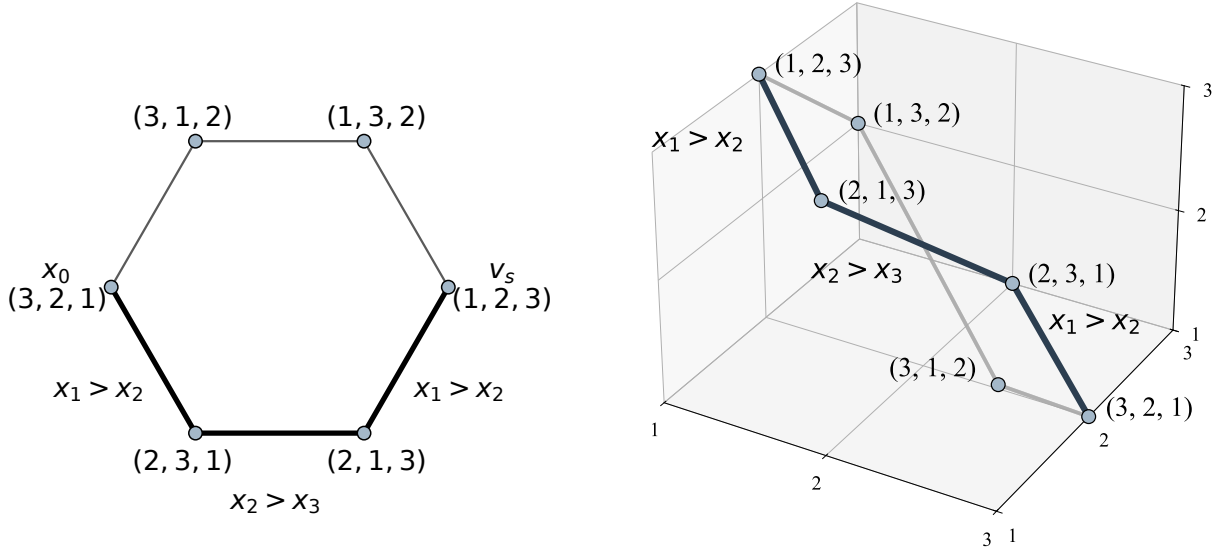


Figure 2: Adjacent-swap motion on the permutohedron  $\mathcal{P}_3$ . **Left:** A 2D schematic of local traversal along the 1-skeleton. **Right:** The same rank-word path  $(3, 2, 1) \rightarrow (2, 3, 1) \rightarrow (2, 1, 3) \rightarrow (1, 2, 3)$  in a 3D embedding. Each step exchanges neighboring positions, illustrating the local  $\Theta(n^2)$  geometry of adjacent-swap sorting.

Let  $C$  denote the comparison outcomes along a fixed decision-tree path. The feasible candidate rank words are

$$\mathcal{V}_n(C) = \{x \in \mathcal{V}_n : x \text{ satisfies every current-position inequality in } C\}. \quad (18)$$

The sorted rank word  $v_s = (1, 2, \dots, n)$  satisfies the final coordinate ordering  $x_1 < x_2 < \dots < x_n$ , but the comparisons that identify it need not be these fixed coordinate tests; they are chosen adaptively by the algorithm.

In the decision-tree model, each comparison has only two possible outcomes. Thus, at best, a comparison can distinguish between two classes of remaining candidate rank words. For the purpose of proving a lower bound, we may therefore use the optimally balanced decision-tree ideal, in which each comparison splits the remaining feasible candidates as evenly as possible. This is the most optimistic binary refinement available to any comparison-based algorithm: not every comparison produces an exactly balanced cut, and arbitrary comparison hyperplanes need not halve either the vertices or the volume of the permutohedron.

In this ideal halving model, after  $k$  comparisons,

$$\frac{n!}{2^k} \leq 1 \quad \Rightarrow \quad k \geq \log(n!) = \Omega(n \log n). \quad (19)$$

Classical algorithms achieve  $O(n \log n)$  comparisons. Hence their constraints cut down the candidate rank words to exactly one feasible vertex. In the final output coordinate frame this vertex is  $v_s$ , so

$$\mathcal{V}_n(C) = \{v_s\}. \quad (20)$$

□

Figure 3 depicts this decision-tree halving heuristic geometrically. Rather than viewing comparisons only as logical tests, each one appears as a half-space slice through  $\mathcal{P}_n$  that can remove candidate rank words.

The feasible region shrinks geometrically from the full permutohedron to a progressively smaller polytope and finally to a single vertex representing the sorted rank word. In this view, the lower bound acquires a concrete geometric form. In the optimally balanced decision-tree ideal, exponential reduction in permutation count is represented by contraction of the feasible set within  $\mathcal{P}_n$ , driven by the global structure imposed by comparisons. Geometrically, this is exactly what happened to the concrete example [5, 4, 2]: the reverse rank word (3, 2, 1) is progressively eliminated by the current-position rank constraints  $x_1 < x_2$  and  $x_2 < x_3$  until only the sorted vertex (1, 2, 3), corresponding to the output [2, 4, 5], remains feasible. These inequalities are constraints on candidate rank words, not fixed claims that the original values  $\ell_1, \ell_2, \ell_3$  already appear in sorted order.

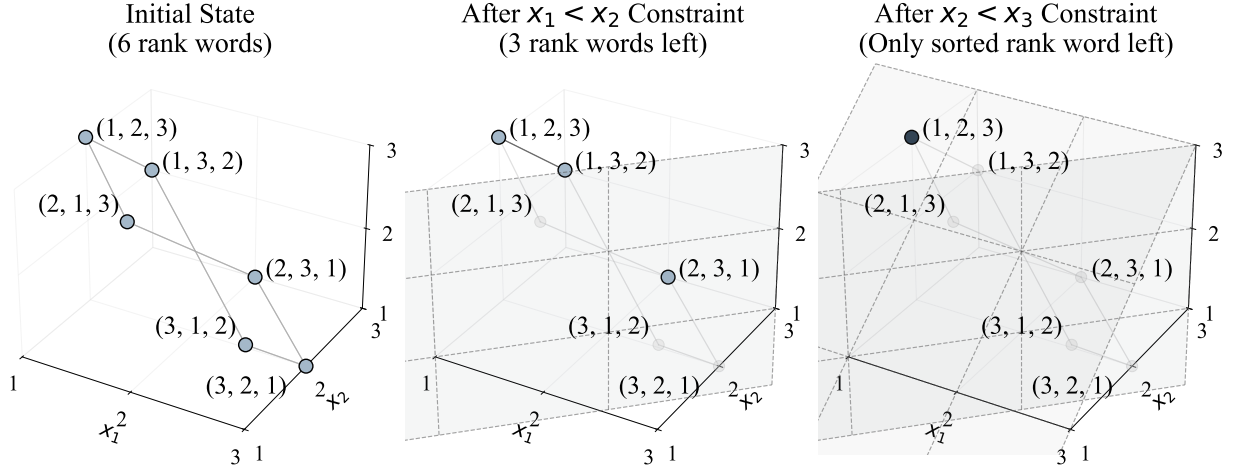


Figure 3: Comparison constraints on  $\mathcal{P}_3$ . The panels show the full rank-word space, the half-space cut  $x_1 < x_2$ , and the subsequent cut  $x_2 < x_3$ , isolating the sorted vertex for  $L = [5, 4, 2]$ . The schematic represents the balanced decision-tree ideal; an actual comparison need not remove a constant fraction of the remaining feasible rank words.

Similar structure-complexity links also appear in work by Blelloch and Dobson, who connect comparison-based sorting to offline binary search trees via a planar representation of permutations and search sequences. Their log-interleave bound offers an information-theoretic lens on sorting cost [5]. While their framework targets BST permutations, our viewpoint treats comparisons as half-space constraints on the permutohedron, emphasizing a general contraction mechanism.

A related geometric approach is explored by Lee et al., who study stack-sorting algorithms via subpolytopes of the permutohedron called stack-sorting simplices [8]. These structures capture how the stack-sort process traverses faces and edges of the permutohedron to identify sortable permutations. Their work provides a detailed case study of structured decomposition within a constrained algorithm. In contrast, the present analysis considers general comparison sorts and treats comparisons as half-space constraints that carve through the full permutohedron. This broader perspective builds on such algorithm-specific insights to describe how constraint-based descent efficiently isolates the sorted order across all sorting procedures.

Before passing to continuous time, it is useful to separate the geometries now on the table. A computational model determines which motions are admissible and therefore determines the relevant notion of distance. If only adjacent exchanges are allowed, the natural metric is the graph distance on the 1-skeleton of  $\mathcal{P}_n$ , equivalently the number of inversions that must be removed. If arbitrary comparisons are allowed, the state is better described by the remaining feasible rank words, and disorder is measured by the information still needed to isolate one vertex. The Euclidean geometry of the standard embedding is a third object: a smooth ambient relaxation that makes contraction visible inside the same polytope. The gradient flow below

should be read in this last sense. It does not claim that a sorting algorithm literally follows a Euclidean trajectory; it gives a continuous benchmark against which the comparison and adjacent-swap geometries can be compared.

This half-space perspective suggests a continuous model in which disorder contracts smoothly toward the sorted vertex, inducing a continuous trajectory through rank-word space. We make this precise via a gradient-flow formulation next.

## 4 Continuous Gradient Flow and the Classical $n \log n$ Scale

The half-space characterization invites a dynamical model of contraction. In this section we present a continuous-time formulation of sorting as an ambient gradient flow on the permutohedron. The formulation offers geometric insight into the exponential contraction of rank displacement under simple dynamics, while the comparison model supplies the intrinsic information scale.

### 4.1 Setup and Definitions

Let

$$v_s = (1, 2, \dots, n) \in \mathbb{R}^n \quad (21)$$

denote the vertex of the permutohedron corresponding to the sorted rank word. For the continuous relaxation, measure rank displacement in the standard Euclidean embedding of the permutohedron and define the potential

$$V(x) = \frac{1}{2} \|x - v_s\|^2 = \frac{1}{2} \sum_{i=1}^n (x_i - i)^2. \quad (22)$$

This is a Spearman-type squared rank displacement inherited from the ambient embedding. It is not the only sorting metric: adjacent-swap sorting is naturally measured by inversion distance, while comparison sorting is naturally measured by the information remaining in the feasible vertex set. The role of  $V$  is to provide a smooth benchmark on  $\mathcal{P}_n$ . Note that  $V(v_s) = 0$  and  $V(x) > 0$  for  $x \neq v_s$ . Given an input list  $L$  with original rank map  $\rho$ , we take as initial geometric state its rank word

$$x(0) = x_0 = (\rho(1), \rho(2), \dots, \rho(n)) \in \mathcal{P}_n, \quad (23)$$

so that the gradient flow evolves the rank word of  $L$  toward the sorted vertex  $v_s$ .

### 4.2 Main Theorem and Proof

**Theorem 4.1** (Ambient Euclidean Contraction). *Let  $x(0) \in \mathbb{R}^n$  be the initial rank word corresponding to an unsorted input list and let  $v_s = (1, 2, \dots, n)$  be the sorted vertex of the permutohedron  $\mathcal{P}_n$ . Define*

$$D_0 = \|x(0) - v_s\|^2. \quad (24)$$

*Under the gradient flow*

$$\dot{x}(t) = -\nabla V(x(t)) = v_s - x(t), \quad (25)$$

*the following statements hold:*

1. *The solution satisfies*

$$x(t) = v_s + (x(0) - v_s)e^{-t}, \quad (26)$$

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}. \quad (27)$$

2. For a threshold  $0 < \epsilon < \sqrt{D_0}$ , the condition  $\|x(t) - v_s\| \leq \epsilon$  holds once

$$t \geq \frac{1}{2} \ln\left(\frac{D_0}{\epsilon^2}\right). \quad (28)$$

3. For the reverse rank word  $x(0) = (n, n-1, \dots, 1)$ ,

$$D_0 = \sum_{i=1}^n (n+1-2i)^2 = \frac{n(n^2-1)}{3} = \Theta(n^3), \quad (29)$$

and therefore

$$V(x(0)) = \frac{D_0}{2} = \frac{n(n^2-1)}{6}. \quad (30)$$

For fixed  $\epsilon = \Theta(1)$ , the threshold time is  $t = \Theta(\ln n)$ .

*Proof.* We begin by deriving the gradient flow dynamics directly from the potential  $V$ . A straightforward computation shows that

$$\nabla V(x) = (x_1 - 1, x_2 - 2, \dots, x_n - n). \quad (31)$$

Hence, the gradient flow is given by

$$\dot{x}(t) = -\nabla V(x(t)) = (1 - x_1(t), 2 - x_2(t), \dots, n - x_n(t)). \quad (32)$$

Because the system decouples, each coordinate  $x_i$  satisfies the first-order linear differential equation

$$\dot{x}_i(t) = i - x_i(t), \quad (33)$$

with initial condition  $x_i(0)$ . Its solution is

$$x_i(t) = i + (x_i(0) - i)e^{-t}. \quad (34)$$

Collecting these coordinate solutions, we obtain

$$x(t) = v_s + (x(0) - v_s)e^{-t}, \quad (35)$$

so that

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}. \quad (36)$$

This establishes statement 1.

For statement 2, define the *initial disorder*  $D_0 = \|x(0) - v_s\|^2$ . The system is effectively sorted when

$$\|x(t) - v_s\|^2 = D_0 e^{-2t} \leq \epsilon^2. \quad (37)$$

Taking logarithms yields

$$t \geq \frac{1}{2} \ln\left(\frac{D_0}{\epsilon^2}\right). \quad (38)$$

**Reverse-permutation radius.** If  $x(0) = (n, n-1, \dots, 1)$  then  $x_i(0) - i = n+1-2i$ . The standard centered-square identity gives

$$D_0 = \sum_{i=1}^n (n+1-2i)^2 = \frac{n(n^2-1)}{3} = \Theta(n^3). \quad (39)$$

Consequently the initial potential at the reverse rank word is

$$V(x(0)) = \frac{1}{2} D_0 = \frac{n(n^2-1)}{6}. \quad (40)$$

Hence  $t = \Theta(\ln n)$ .  $\square$

### 4.3 Entropy-Normalized Discrete Time

The passage from continuous dynamics to discrete comparisons is used here as a modeling correspondence, not as a replacement for the decision-tree lower bound. The natural clock for this correspondence is not an arbitrary physical time but comparison information per element.

**Proposition 4.2** (Entropy-normalized comparison time). *After  $k$  binary comparisons, define the normalized comparison time*

$$s_k = \frac{k}{n}. \quad (41)$$

*Equivalently, if information is measured in nats, one may use  $s_k = (k \ln 2)/n$ , which differs only by a constant factor. Then the classical comparison scale  $k = \Theta(n \log n)$  is exactly the normalized scale*

$$s_k = \Theta(\log n). \quad (42)$$

*Thus the continuous threshold time  $t = \Theta(\ln n)$  in Theorem 4.1 lies on the same macroscopic scale as optimal comparison sorting. In this normalization, one comparison advances the macroscopic clock by  $1/n$ .*

*Proof.* Each comparison has two possible outcomes and therefore contributes at most one bit of information about the remaining order type. The decision-tree lower bound gives

$$k \geq \log_2(n!) = n \log_2 n - O(n), \quad (43)$$

while optimal comparison sorts achieve  $k = O(n \log n)$ . Dividing the comparison count by  $n$  gives  $s_k = \Theta(\log n)$ . Since  $s_{k+1} - s_k = 1/n$ , the order- $1/n$  increment is not an independent assumption about the Euclidean flow; it is the microscopic step size induced by normalizing binary comparison information per element.  $\square$

This proposition should be read as a normalization rather than an independent proof of the classical lower bound. The rigorous lower bound remains the decision-tree argument of Section 2; the continuous contraction model supplies a compatible geometric scale once comparison information is measured per element. A fully intrinsic version of this normalization can instead be expressed by a unit-cost Finsler geometry in which each primitive operation has unit length.

### 4.4 Incorporating the Permutohedron Constraints

The exact flow in Theorem 4.1 is an ambient Euclidean benchmark. This benchmark is already compatible with the convex geometry of the permutohedron:  $\mathcal{P}_n$  lies in the affine hyperplane

$$\sum_{i=1}^n x_i = \frac{n(n+1)}{2}, \quad (44)$$

and both  $x(0)$  and  $v_s$  lie in  $\mathcal{P}_n$ . Since  $\mathcal{P}_n$  is convex, the straight segment

$$x(t) = v_s + (x(0) - v_s)e^{-t} \quad (45)$$

remains in  $\mathcal{P}_n$  for all  $t \geq 0$ .

It is important not to identify comparison hyperplanes with the boundary facets of  $\mathcal{P}_n$ . The hyperplanes

$$x_i = x_j \quad (46)$$

are braid-arrangement walls in the ambient affine space; they slice  $\mathcal{P}_n$ , but they are not, in general, facets of the permutohedron. The facets of the standard permutohedron are instead described by subset-sum inequalities of the form

$$\sum_{i \in S} x_i \geq \frac{|S|(|S| + 1)}{2}, \quad \emptyset \neq S \subseteq \{1, \dots, n\}, \quad (47)$$

together with the fixed total-sum equality. Thus projection is not forced merely by the fact that the trajectory lies in  $\mathcal{P}_n$ .

Projected dynamics become relevant after comparison outcomes have imposed additional feasibility constraints. For a family  $C$  of comparison outcomes, let  $\bar{C}$  denote the corresponding closed comparison half-spaces, obtained by replacing strict inequalities between distinct rank-word vertices by their closed versions. This closure does not change the feasible vertex set, but it gives a closed convex set for the projected-flow model. Write

$$K(C) = \mathcal{P}_n \cap \{x : x \text{ satisfies every closed half-space constraint in } \bar{C}\}. \quad (48)$$

On such a convex feasible set, the projected gradient flow is

$$\dot{x}(t) = \Pi_{T_{K(C)}(x(t))}(-\nabla V(x(t))), \quad (49)$$

where  $\Pi_{T_{K(C)}(x(t))}$  denotes Euclidean projection onto the tangent cone of  $K(C)$  at  $x(t)$ . Along this constrained flow,

$$\frac{d}{dt}V(x(t)) = \langle \nabla V(x(t)), \dot{x}(t) \rangle = -\|\Pi_{T_{K(C)}(x(t))}(-\nabla V(x(t)))\|^2 \leq 0. \quad (50)$$

Since projection can only decrease norm,

$$\|\Pi_{T_{K(C)}(x(t))}(-\nabla V(x(t)))\| \leq \|\nabla V(x(t))\|, \quad (51)$$

and therefore

$$\frac{d}{dt}V(x(t)) \geq -\|\nabla V(x(t))\|^2 = -2V(x(t)). \quad (52)$$

The unconstrained decay law

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t} \quad (53)$$

therefore serves as a fastest-benchmark comparison: adding convex comparison constraints can slow the instantaneous decrease of  $V$ , but it cannot make the projected constrained flow contract  $V$  faster than the ambient Euclidean flow. The well-posedness of such projected flows follows from standard results on projection onto convex tangent cones (see [9]).

## 4.5 Remarks

Theorem 4.1 demonstrates that continuous gradient descent under the potential  $V(x) = \frac{1}{2}\|x - v_s\|^2$  gives exact exponential contraction of ambient rank displacement. Proposition 4.2 then places this motion on an entropy-normalized comparison clock: one comparison has size  $1/n$ , and the classical comparison scale becomes  $\Theta(\log n)$ . The decision-tree proof remains the rigorous lower-bound argument; the flow supplies a compatible geometric benchmark. In this respect the viewpoint echoes the insight of Jordan, Kinderlehrer, and Otto, who reinterpreted the Fokker–Planck equation as a gradient flow minimizing a free energy functional under the Wasserstein metric [10].

Figure 4 completes the transition from discrete comparisons to continuous geometry. Vertices represent rank words, and the gradient flow traces a smooth ambient trajectory inside the convex permutohedron toward

the sorted vertex. The left panel contrasts this trajectory with the adjacent-swap path: local algorithms advance by short moves along edges, while the ambient relaxation cuts across the polytope by straight-line descent. This path is not a graph-geodesic on the 1-skeleton and is not the literal trajectory of a discrete sorting algorithm.

The right panel presents the same motion in scalar form. The distance to the sorted state contracts exponentially as  $r(t) = \|x(t) - v_s\|_2$  decreases. Viewed in  $(r, t, V)$ -space, the trajectory resembles motion through a curved landscape: the system rolls downhill under the potential, like a particle descending a gravitational well toward equilibrium. In both panels the example is the same list  $L = [5, 4, 2]$  used earlier: the rank word begins at  $(3, 2, 1)$  and flows toward  $(1, 2, 3)$ , corresponding to the sorted output  $[2, 4, 5]$ . The initial radius is  $r_0 = \|(3, 2, 1) - (1, 2, 3)\|_2 = \sqrt{8}$ , and the initial dissipation is  $dV/dt = -\|\nabla V(x(0))\|_2^2 = -8$ .

Together, the panels show the role of the relaxation without overstating it. Comparisons still provide the discrete information, but the continuous model shows what that information is organizing: a structured space in which disorder can be measured, contracted, and compared across different geometries. The classical  $\Theta(n \log n)$  scale is therefore consistent with logarithmic contraction on the entropy-normalized clock.

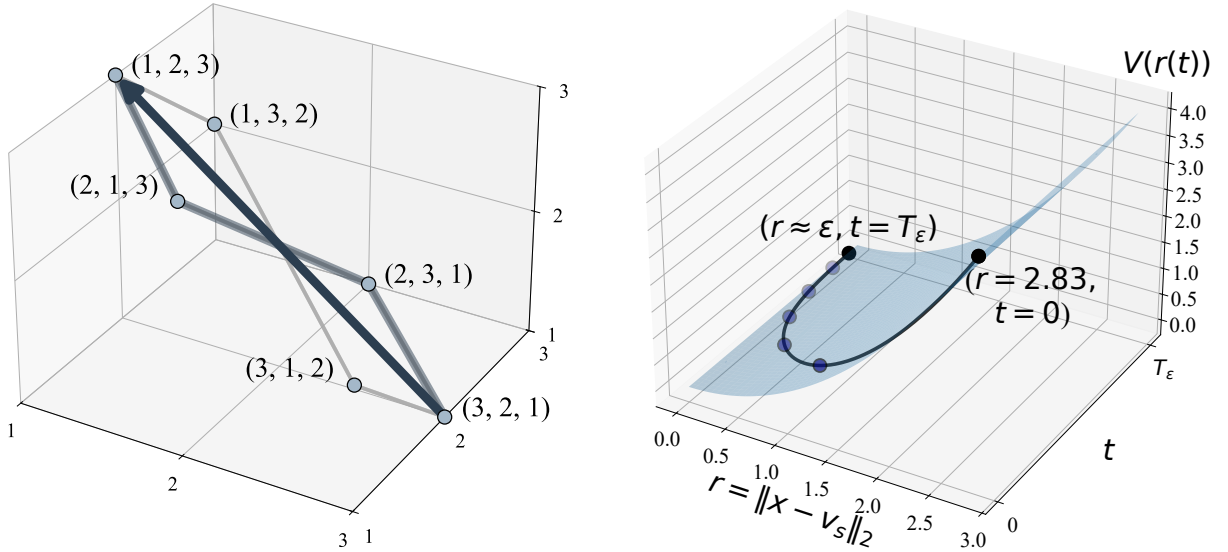


Figure 4: Gradient-flow relaxation under  $V(x) = \frac{1}{2}\|x - v_s\|_2^2$ . **Left:** Straight-line ambient descent on  $\mathcal{P}_3$  from  $(3, 2, 1)$  to  $(1, 2, 3)$ , contrasted with the adjacent-swap path. **Right:** Exponential contraction of  $r(t) = \|x(t) - v_s\|_2$  from  $r_0 = \sqrt{8}$  to a threshold  $\epsilon$ . This is a Euclidean benchmark, not the literal path of a discrete sorting algorithm; under entropy-normalized comparison time, its  $\Theta(\log n)$  contraction scale matches the  $\Theta(n \log n)$  comparison scale.

## 5 Conclusion

The central challenge in efficient sorting is overcoming the factorial scale of the permutation space. Established algorithms do this by exploiting structure rather than by searching faster. MergeSort uses the recurrence

$$T(n) = 2T(n/2) + O(n) \quad (54)$$

to obtain an  $O(n \log n)$  running time through divide-and-conquer, and von Neumann’s implementation showed how recursive design could yield both theoretical and practical efficiency [11]. QuickSort uses pivot choices to prune the search space in expectation; HeapSort imposes heap structure. In each case, the algorithm succeeds by organizing the permutation space before it is exhausted.

From a geometric perspective, this distinction appears as the contrast between local motion and global geometry. Adjacent-swap algorithms traverse the permutohedron along its edges, resolving one inversion at a time and producing the classical  $\Theta(n^2)$  behavior. Arbitrary comparisons act differently: they impose constraints that reshape the feasible region as a whole. This is the geometric meaning of the decision-tree improvement. Sorting succeeds not by walking the polytope faster, but by collapsing it. This theme is consistent with structured decomposition methods in combinatorial algorithms [12] and with recent enumerative work of Harris, Kretschmann, and Martínez Mori, where QuickSort comparisons are counted by parking preference lists with  $n - 1$  “lucky cars” [6].

In parallel, we introduced a continuous-time formulation that recasts sorting as an ambient gradient flow on the permutohedron. The model is a relaxation, not the literal path of a sorting algorithm, but it clarifies the geometries at play. Adjacent swaps give the graph geometry of the 1-skeleton. Arbitrary comparisons give an information geometry on feasible rank words. The Euclidean flow gives a smooth rank-displacement benchmark inside the same polytope. As the flow contracts the ambient distance to the sorted order according to

$$\|x(t) - v_s\|^2 = \|x(0) - v_s\|^2 e^{-2t}, \quad (55)$$

time becomes a measure of geometric convergence. Discrete complexity re-enters through comparison information: one binary comparison advances the normalized clock by  $1/n$ , and the decision-tree lower bound forces  $\Theta(\log n)$  units of macroscopic time.

Shannon’s information-theoretic analysis made clear that search is governed by available information [4]. Complexity theory develops this principle across many computational settings [13]; Mann’s perspective on search emphasizes the way constraints organize rather than merely restrict a space [14]. The present model gives this idea a concrete geometric form for sorting. Factorial search collapses into log-linear time when comparisons supply enough global information to organize the feasible rank words. In that sense, local motion and global structure meet inside the same object: the permutohedron, its comparison half-spaces, and the ambient flow toward order.

## Acknowledgment

The author used standard software tools, programming libraries, and AI-assisted computational tools to support figure preparation, algebraic checking, and editorial refinement. All mathematical content, interpretations, and final wording were conceptualized and finalized by the author.

**Disclosure of interest.** The author reports no conflict of interest.

**Funding.** No funding was received for this research.

## References

- [1] Knuth DE. Big Omicron and Big Omega and Big Theta. SIGACT News. 1976;8(2):18-24.
- [2] Cormen TH, Leiserson CE, Rivest RL, Stein C. Introduction to Algorithms. 4th ed. Cambridge, MA: MIT Press; 2022.

- [3] Knuth DE. The art of computer programming, volume 3: sorting and searching. Reading, MA: Addison-Wesley; 1973.
- [4] Shannon CE. A mathematical theory of communication. Bell Syst Tech J. 1948;27:379-423, 623-56. Parts I–II; part I doi: 10.1002/j.1538-7305.1948.tb01338.x; part II doi: 10.1002/j.1538-7305.1948.tb00917.x.
- [5] Blelloch GE, Dobson M. The geometry of tree-based sorting. In: Proceedings of the 50th International Colloquium on Automata, Languages, and Programming (ICALP 2023). vol. 261 of LIPIcs. Paderborn, Germany: Schloss Dagstuhl–Leibniz-Zentrum für Informatik; 2023. p. 26:1-26:19. Available from: <https://doi.org/10.4230/LIPIcs.ICALP.2023.26>.
- [6] Harris PE, Kretschmann J, Martínez Mori JC. “Lucky cars” and the quicksort algorithm. The American Mathematical Monthly. 2024;131(5):417-23.
- [7] Ziegler GM. Lectures on polytopes. New York, NY: Springer-Verlag; 1995.
- [8] Lee E, Mitchell C, Vindas-Meléndez AR. Stack-sorting simplices: geometry and lattice-point enumeration. arXiv preprint. 2023. ArXiv:2308.16457.
- [9] Rockafellar RT. Convex analysis. Princeton, NJ: Princeton University Press; 1970.
- [10] Jordan R, Kinderlehrer D, Otto F. The variational formulation of the Fokker–Planck equation. SIAM J Math Anal. 1998;29(1):1-17.
- [11] von Neumann J. First draft of a report on the EDVAC. Philadelphia, PA: Moore School of Electrical Engineering, University of Pennsylvania; 1945.
- [12] Hopcroft JE, Tarjan RE. Algorithm 447: efficient algorithms for graph manipulation. Commun ACM. 1973;16(6):372-8.
- [13] Arora S, Barak B. Computational complexity: a modern approach. Cambridge University Press; 2009.
- [14] Mann Z. The top eight misconceptions about NP-hardness. Computer. 2017;50(5):72-9.